ASSIGNMENT 4

Relational Database vs. NoSQL Database

Project: Airline Reservation System | Course: Database Management Systems

Objectives covered

- Compare relational and NoSQL databases: scalability, consistency, performance and use cases
- Explain why organizations choose different database technologies
- Present a comparative report with real-world examples, applied to the Airline Reservation System

1. Introduction

Every application must decide how to store its data. For about four decades the answer was almost always a **relational database management system (RDBMS)** such as Oracle, MySQL, PostgreSQL or SQL Server. In the 2000s, internet companies handling enormous numbers of users and rapidly changing data formats built a new family of systems, collectively called **NoSQL** ("not only SQL"), such as MongoDB, Cassandra, Redis, DynamoDB and Neo4j. Neither family is better in every situation; they make different trade-offs. This report compares them on **scalability, consistency, performance and use cases**, explains why organizations pick one or the other, and applies the comparison to our **Airline Reservation System**.

2. Overview of the Two Families

2.1 Relational databases (RDBMS)

Data is stored in **tables** (relations) made of rows and columns. The structure (schema) is defined up front, relationships are expressed with primary and foreign keys, data is queried with the standard language **SQL**, and transactions follow the **ACID** properties: **A**tomicity (all or nothing), **C**onsistency (constraints always hold), **I**solation (concurrent transactions do not interfere) and **D**urability (committed data survives crashes). Our project database (eight normalized tables) is a relational design.

2.2 NoSQL databases

NoSQL covers several different data models, each optimized for a particular access pattern. Most are designed to **scale horizontally** across many cheap servers and to accept flexible (schema-less or schema-on-read) data. Many follow the **BASE** philosophy: **B**asically **A**vailable, **S**oft state, **E**ventually consistent.

Type	How data is stored	Typical products	Best at
Document	JSON/BSON documents with nested fields, grouped in collections	MongoDB, Couchbase, Firestore	Catalogues, user profiles, content, nested data
Key-value	A key mapped to an opaque value; direct lookup by key	Redis, Amazon DynamoDB, Memcached	Caching, sessions, shopping carts, counters
Wide-column	Rows with dynamic column families, partitioned by key	Apache Cassandra, HBase, Bigtable	Huge write volumes, time-series, event logs
Graph	Nodes and edges with properties	Neo4j, Amazon Neptune	Relationship-heavy queries: routes, fraud, recommendations

3. Detailed Comparison

3.1 Scalability

Relational. Traditionally scales **vertically** (scale-up): a bigger server with more CPU, RAM and faster disks. This is simple and keeps joins and transactions fast, but there is a hardware ceiling and cost grows steeply. Reads can be scaled by adding *read replicas*, and large tables can be *partitioned or sharded*, but sharding a relational schema is complicated: joins and transactions that span shards are expensive, and the application usually has to manage the shard logic.

NoSQL. Most NoSQL systems were built for **horizontal scaling** (scale-out): data is automatically partitioned (sharded) across a cluster, and capacity grows by adding commodity nodes. Cassandra and DynamoDB, for example, distribute data by a partition key and replicate it across nodes and data centres. The price is that operations spanning several partitions (joins, multi-record transactions) are limited or absent.

3.2 Consistency

Relational: strong consistency by default. After a transaction commits, every subsequent reader sees the result; constraints (PRIMARY KEY, FOREIGN KEY, CHECK) are enforced by the engine itself. **NoSQL:** consistency is usually *tunable*. Many systems favour availability and speed and replicate data asynchronously, so for a short time two replicas can return different values (**eventual consistency**). This behaviour is explained by the **CAP theorem**: when a network partition occurs, a distributed system must choose between **Consistency** and **Availability**; it cannot guarantee both.

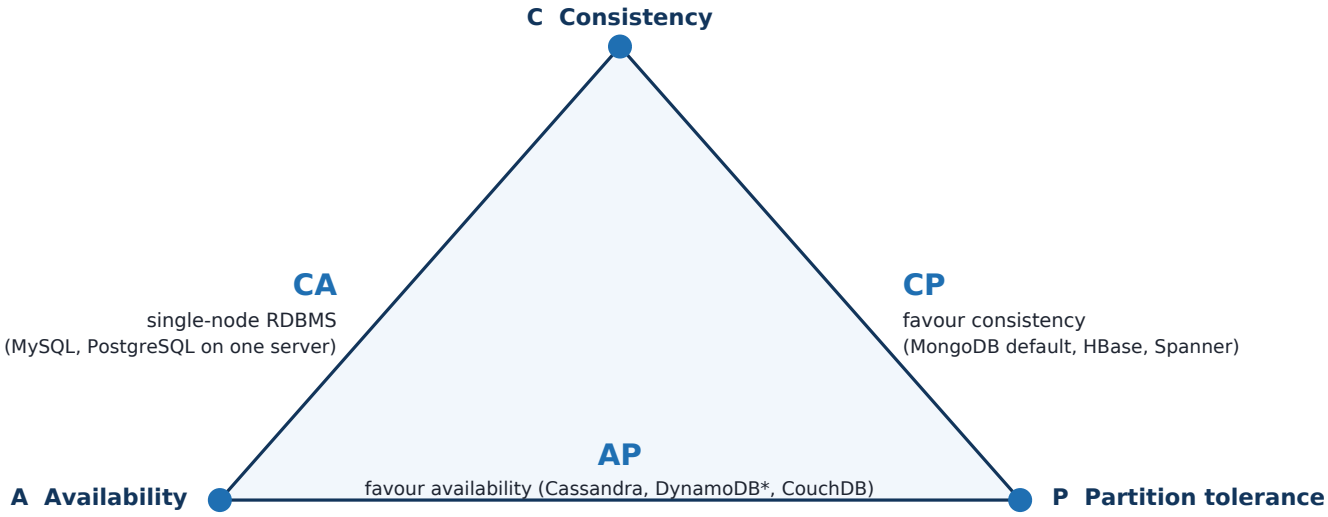


*Figure 1: The CAP theorem, simplified. *Placement is indicative: many products are configurable (for example DynamoDB and Cassandra both offer strongly consistent reads at a cost).*

3.3 Performance

Performance depends on the workload, not on the label. **Relational** engines are very fast for complex queries, joins, aggregation and multi-row transactions, thanks to mature query optimizers and indexes; they can slow down when a single table grows to billions of rows or when many joins are required at very high request rates. **NoSQL** systems are very fast for simple access patterns such as *get by key*, *write an event* or *read one whole document*, often with predictable single-digit-millisecond latency at huge scale, because the data is denormalized (stored together) and no join is needed. They are slower or unsuitable for ad-hoc queries, reporting and cross-entity analysis unless the data model was designed for that query in advance.

3.4 Use cases

Relational databases fit **structured, highly related data with strict integrity needs**: banking, billing, inventory, ERP/CRM, reservations. NoSQL fits **very large, fast-growing, semi-structured or loosely related data**: social feeds, IoT and telemetry, caching, product catalogues, real-time personalization, and graph-style relationship analysis.

3.5 Side-by-side summary

Criterion	Relational database (RDBMS)	NoSQL database
Data model	Tables, rows, columns; relationships by foreign keys	Documents, key-value pairs, wide columns or graphs
Schema	Fixed, defined before data is stored (schema-on-write)	Flexible / dynamic (schema-on-read)
Query language	Standard SQL (joins, GROUP BY, subqueries)	Product-specific APIs / query languages (e.g. MongoDB query, CQL, Cypher)
Scalability	Mainly vertical; horizontal via replication / sharding with effort	Designed for horizontal scale-out across commodity servers
Consistency	Strong; ACID transactions	Tunable; often eventual (BASE); transactions limited or per-document
Relationships and joins	Native, enforced by foreign keys	Embedded documents or manual references; joins limited or absent
Data integrity	Constraints enforced by the database	Mostly enforced by the application
Performance strengths	Complex queries, aggregation, multi-table transactions	Simple, high-volume reads/writes; low latency at scale
Redundancy	Minimized by normalization	Accepted (denormalization) to avoid joins
Maturity / tools	Very mature; standard tooling, skills widely available	Newer; varied tooling; skills specific to each product
Typical examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Cassandra, Redis, DynamoDB, Neo4j

4. Why Organizations Choose Different Database Technologies

There is no universally best database; the choice follows the **requirements** of the application. The main decision factors are:

Factor	Questions asked	Leans towards
Data structure	Is it tabular and well-defined, or varied and changing?	Structured → RDBMS; flexible/nested → Document NoSQL
Consistency needs	Can two users ever see different values (money, seats)?	Must never → RDBMS (ACID); can tolerate delay → NoSQL
Scale and growth	Millions of users? Millions of writes per second? Global?	Massive or unpredictable → NoSQL / distributed
Query patterns	Ad-hoc reports and joins, or a few known lookups?	Ad-hoc, analytical → SQL; known key lookups → key-value / wide-column
Relationships	Are connections themselves the data (routes, networks)?	Highly connected → Graph; moderately → RDBMS
Speed of change	Does the data model change every sprint?	Rapid change → Document store
Compliance and audit	Regulated data needing traceability and integrity?	RDBMS (mature auditing, constraints)
Team skills and ecosystem	Which technology does the team know? Which tools exist?	SQL skills are widespread; NoSQL needs product expertise
Cost and operations	Licence, cloud service or in-house cluster management?	Managed cloud services reduce the burden for both

Because the requirements differ between parts of one application, large organizations commonly practise **polyglot persistence**: using several databases, each for the job it does best. A further trend is **NewSQL** (for example Google Cloud Spanner, CockroachDB, TiDB), which tries to combine SQL and ACID

transactions with horizontal scalability.

5. Real-World Examples

Organization / system	Technology	Why it fits (public information)
Banks and payment processors (core banking, ledgers)	Relational: Oracle, PostgreSQL, SQL Server, DB2	Money transfers must be atomic and consistent; regulators require audit trails and constraints.
Amazon (shopping cart, order pipeline)	Amazon DynamoDB (key-value), originally the 'Dynamo' system	Amazon described Dynamo as a highly available key-value store so that the cart stays writable even during failures; availability over strict consistency.
Facebook (inbox search, 2008)	Apache Cassandra (wide-column), created at Facebook and later open-sourced	Needed very high write throughput and scalability across many servers for messaging data.
Netflix	Apache Cassandra for large-scale, multi-region data (viewing history and similar), alongside other stores	Needs availability across regions and the ability to add nodes without downtime.
Stack Overflow	SQL Server (primary data) plus Redis (caching)	Shows the common pattern: a relational system of record with a fast in-memory cache in front.
Instagram	PostgreSQL for core data and Cassandra for some large-volume use cases	A typical polyglot setup, described in the company's engineering blogs.
Uber	MySQL-based storage layers built to scale (and other systems)	Even a hyper-scale company built its scalable storage on top of a relational engine.
eBay and many e-commerce sites	MongoDB and similar document stores for catalogue and content data	Products have different attributes (a phone and a shirt); documents avoid many sparse columns.
Fraud detection and recommendation engines	Graph databases such as Neo4j	Queries such as 'who is connected to whom through shared cards or devices' are natural graph traversals.
Google (global transactional systems)	Cloud Spanner (NewSQL)	Offers SQL with strongly consistent transactions at global scale.

Examples are drawn from publicly available engineering articles, research papers and vendor case studies; exact internal architectures change over time.

6. Application to the Airline Reservation System

An airline system is a good example of **why one size does not fit all**. Different components have different requirements:

Component	Requirement	Suitable technology	Reason
Bookings, tickets, payments, refunds	Absolute correctness; no two passengers on one seat; money must balance	**Relational (our project)**	ACID transactions, UNIQUE (flight, seat) and foreign keys enforce rules in the database itself.
Flight and fare search results	Millions of repeated look-ups; results may be a few minutes old	Redis cache / search engine such as Elasticsearch	Very fast key lookups; staleness is acceptable until the final booking step re-checks the RDBMS.
Passenger preferences, profiles, loyalty data	Flexible fields that change often	Document store (MongoDB)	One document per customer holds nested preferences, with no schema migration for new fields.
Clickstream, mobile-app events, aircraft sensor logs	Enormous write volume, time-ordered	Wide-column (Cassandra)	Write-optimized, scales out by adding nodes.
Route networks, connecting itineraries, fraud rings	Relationship traversal	Graph (Neo4j)	Finding 'all routes of at most 2 stops from BLR to SIN' is a graph path query.
Reporting and analytics	Aggregation across all data	Data warehouse fed from the RDBMS	SQL GROUP BY, joins and window functions.

6.1 The same booking: relational vs. document model

In the relational design a booking is spread over four tables and re-assembled with joins (see Assignment 9). In a document database the same booking would be one self-contained, denormalized document. The illustration below is for comparison only; it is not part of the implemented database.

Relational query (illustrative)

```
-- Relational (implemented): four normalized tables, rebuilt with JOINs
SELECT b.booking_ref, p.first_name, f.flight_number, t.seat_no, t.fare, pay.status
FROM booking b
JOIN passenger p ON b.passenger_id = p.passenger_id
JOIN ticket t    ON t.booking_id   = b.booking_id
JOIN flight f    ON t.flight_id    = f.flight_id
JOIN payment pay ON pay.booking_id = b.booking_id
WHERE b.booking_ref = 'XXXXXX';
```

MongoDB-style document (illustrative)

```
{
  "_id": "booking_3",
  "bookingRef": "XXXXXX",
  "bookedBy": { "passengerId": 3, "name": "Rohan Mehta", "email": "rohan.mehta@example.com" },
  "tickets": [
    { "flight": { "number": "6E-501", "from": "BLR", "to": "DEL", "departs": "2026-11-02T06:00" },
      "passenger": "Rohan Mehta", "seat": "8A", "class": "Economy", "fare": 4680 },
    { "flight": { "number": "6E-501", "from": "BLR", "to": "DEL", "departs": "2026-11-02T06:00" },
      "passenger": "Ananya Iyer", "seat": "8B", "class": "Economy", "fare": 4680 } ],
  "payments": [ { "method": "Wallet", "amount": 9360, "status": "Success" } ],
  "total": 9360
}
```

Aspect	Relational design	Document design
Reading one booking	Several joins	One read: everything is already together (faster)
Flight time changes	Update 1 row in FLIGHT; all tickets see it	Flight data is copied inside every booking document; each must be updated (redundancy and update anomaly)
Report: revenue by airline	Simple JOIN + GROUP BY	Needs aggregation pipeline over copied data; harder
Seat uniqueness per flight	UNIQUE (flight_id, seat_no) enforced by the database	Must be coded in the application; race conditions possible

This comparison shows why the core reservation data of this project is relational: it is highly interrelated, it changes in place (flight delays, cancellations), and it needs the integrity guarantees (uniqueness of seats, foreign keys, atomic payment and refund) that the relational model provides out of the box.

7. Advantages and Limitations at a Glance

	Advantages	Limitations
RDBMS	Strong consistency and ACID; powerful SQL; constraints and integrity; mature tools; ideal for reporting	Harder to scale out; rigid schema changes; joins become costly at extreme scale; object-relational mismatch
NoSQL	Horizontal scale and high availability; flexible schema; very fast simple operations; model matched to the use case	Weaker consistency guarantees; limited joins/transactions; no standard query language; integrity is the application's job; data duplication

8. Conclusion

Relational and NoSQL databases are complementary, not rivals. Relational systems offer strong consistency, integrity and expressive SQL, which makes them the right choice for transactional, interrelated data such as reservations and payments. NoSQL systems trade some of that rigidity for horizontal scalability, flexibility and speed on specific access patterns, which suits caching, event streams, flexible profiles and graph analysis. Organizations choose according to data structure, consistency requirements, scale, query patterns, skills and cost, and large systems frequently combine several databases. For this Airline Reservation System, the transactional core (flights, bookings, tickets and payments) is implemented in a relational database, while a production deployment could add a cache and search layer, a document store for profiles and a column store for event logs around it.

References

- DeCandia, G. et al. (2007). *Dynamo: Amazon's Highly Available Key-value Store.* ACM SOSP.
- Lakshman, A. and Malik, P. (2010). *Cassandra: A Decentralized Structured Storage System.* ACM SIGOPS Operating Systems Review.
- Corbett, J. et al. (2012). *Spanner: Google's Globally-Distributed Database.* USENIX OSDI.
- Brewer, E. (2000). *Towards Robust Distributed Systems* (the CAP conjecture); Gilbert and Lynch (2002) formal proof.
- Silberschatz, Korth and Sudarshan. *Database System Concepts*. McGraw-Hill.
- Official documentation: MongoDB, Apache Cassandra, Redis, Amazon DynamoDB, Neo4j, PostgreSQL, MySQL.